第四次作业：SVHN 模型的 INT8 静态量化

实验目的

本实验在 SVHN 街景数字分类任务上完成训练后静态量化（Post-Training Static Quantization, PTQ）。实验首先训练或加载 FP32 CNN 基线模型，然后手动实现 per-tensor 非对称线性量化与反量化，最后利用 PyTorch 完成量化节点插入、模块融合、校准和 INT8 模型转换。

实验将从以下四个方面比较 FP32 与 INT8 模型：

1. 测试集准确率与量化精度损失；
2. 模型文件大小与压缩比；
3. CPU 单张图像平均推理延迟与加速比；
4. 各主要网络层输出的量化均方误差（MSE）。

> 说明：INT8 静态量化模型在 CPU 上运行。若当前目录已有 `svhn_fp32_cnn.pth`，程序会直接加载；否则会自动训练并保存 FP32 模型。提交作业时不需要包含下载的 SVHN 数据集。

1. 环境配置与随机种子

建议使用较新的 PyTorch 与 torchvision。若缺少依赖，可在当前环境执行：

```
pip install torch torchvision matplotlib pandas
```

In [1]:
```
import copy
import io
import os
import random
import time
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim
from torch.ao.quantization import (
    DeQuantStub,
    MinMaxObserver,
    QConfig,
    QuantStub,
    convert,
    fuse_modules,
    prepare,
)
```

```
from torch.utils.data import DataLoader, Subset
from torchvision import datasets, transforms

SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)

plt.rcParams["font.sans-serif"] = ["SimHei", "Microsoft YaHei", "Arial Unicode M
plt.rcParams["axes.unicode_minus"] = False

TRAIN_DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
CPU_DEVICE = torch.device("cpu")
DATA_DIR = Path("./data")
CHECKPOINT_PATH = Path("./svhn_fp32_cnn.pth")

BATCH_SIZE = 128
TEST_BATCH_SIZE = 256
NUM_EPOCHS = 15
LEARNING_RATE = 1e-3
CALIBRATION_SAMPLES = 1024

print("PyTorch 版本：", torch.__version__)
print("FP32 训练设备：", TRAIN_DEVICE)
print("量化后端列表：", torch.backends.quantized.supported_engines)
```

```
PyTorch 版本： 2.8.0+cu129
FP32 训练设备： cuda
量化后端列表： ['none', 'onednn', 'x86', 'fbgemm']
```

2. SVHN 数据集与预处理

SVHN 数据集由 32×32 的 RGB 门牌数字图像组成，共 10 个类别。训练阶段使用随机裁剪和轻微旋转增强泛化能力；测试与校准阶段仅进行张量化和标准化，以保证结果稳定。校准集从训练集前 `CALIBRATION_SAMPLES` 个样本中抽取，不使用标签参与校准。

In [2]:
```
SVHN_MEAN = (0.4377, 0.4438, 0.4728)
SVHN_STD = (0.1980, 0.2010, 0.1970)

train_transform = transforms.Compose([
    transforms.RandomCrop(32, padding=4),
    transforms.RandomRotation(8),
    transforms.ColorJitter(brightness=0.15, contrast=0.15),
    transforms.ToTensor(),
    transforms.Normalize(SVHN_MEAN, SVHN_STD),
])

eval_transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(SVHN_MEAN, SVHN_STD),
])

train_dataset = datasets.SVHN(
    root=DATA_DIR, split="train", download=True, transform=train_transform
)
calibration_source = datasets.SVHN(
```

```
    root=DATA_DIR, split="train", download=True, transform=eval_transform
)
test_dataset = datasets.SVHN(
    root=DATA_DIR, split="test", download=True, transform=eval_transform
)

generator = torch.Generator().manual_seed(SEED)
calibration_indices = torch.randperm(
    len(calibration_source), generator=generator
)[: min(CALIBRATION_SAMPLES, len(calibration_source))].tolist()
calibration_dataset = Subset(calibration_source, calibration_indices)

pin_memory = torch.cuda.is_available()
train_loader = DataLoader(
    train_dataset, batch_size=BATCH_SIZE, shuffle=True,
    num_workers=0, pin_memory=pin_memory
)
calibration_loader = DataLoader(
    calibration_dataset, batch_size=128, shuffle=False, num_workers=0
)
test_loader = DataLoader(
    test_dataset, batch_size=TEST_BATCH_SIZE, shuffle=False, num_workers=0
)

print(f"训练集样本数：{len(train_dataset):,}")
print(f"校准集样本数：{len(calibration_dataset):,}")
print(f"测试集样本数：{len(test_dataset):,}")
```

```
100%|██████████| 182M/182M [00:15<00:00, 11.8MB/s]
100%|██████████| 64.3M/64.3M [00:18<00:00, 3.46MB/s]
训练集样本数：73,257
校准集样本数：1,024
测试集样本数：26,032
```

In [3]:

```
# 显示部分训练样本
images, labels = next(iter(train_loader))
mean = torch.tensor(SVHN_MEAN).view(3, 1, 1)
std = torch.tensor(SVHN_STD).view(3, 1, 1)

fig, axes = plt.subplots(2, 8, figsize=(13, 4))
for i, ax in enumerate(axes.flat):
    image = (images[i].cpu() * std + mean).permute(1, 2, 0).clamp(0, 1)
    ax.imshow(image)
    ax.set_title(f"标签：{labels[i].item()}")
    ax.axis("off")
plt.suptitle("SVHN 训练样本")
plt.tight_layout()
plt.show()
```

3. 构建可静态量化的 CNN

模型由四个卷积块和两层全连接层组成。为满足静态量化要求，在输入端和输出端分别加入 `QuantStub` 与 `DeQuantStub`：

- `QuantStub`：将归一化后的 FP32 输入转换到量化域；
- `DeQuantStub`：将最终分类 logits 从量化域恢复为 FP32；
- 卷积块中的 `Conv2d + BatchNorm2d + ReLU` 被融合；
- 分类器中的 `Linear + ReLU` 被融合。

模块融合既减少了运行时算子数量，也能避免中间张量反复量化造成额外误差。

In [4]:
```
class ConvBlock(nn.Module):
    def __init__(self, in_channels, out_channels):
        super().__init__()
        self.conv1 = nn.Conv2d(
            in_channels, out_channels, kernel_size=3, padding=1, bias=False
        )
        self.bn1 = nn.BatchNorm2d(out_channels)
        self.relu1 = nn.ReLU(inplace=False)
        self.conv2 = nn.Conv2d(
            out_channels, out_channels, kernel_size=3, padding=1, bias=False
        )
        self.bn2 = nn.BatchNorm2d(out_channels)
        self.relu2 = nn.ReLU(inplace=False)
        self.pool = nn.MaxPool2d(kernel_size=2, stride=2)

    def forward(self, x):
        x = self.relu1(self.bn1(self.conv1(x)))
        x = self.relu2(self.bn2(self.conv2(x)))
        return self.pool(x)

    def fuse_model(self):
        fuse_modules(
            self,
            [["conv1", "bn1", "relu1"], ["conv2", "bn2", "relu2"]],
            inplace=True,
        )


class QuantizableSVHNCNN(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        self.quant = QuantStub()
        self.block1 = ConvBlock(3, 32)
        self.block2 = ConvBlock(32, 64)
        self.block3 = ConvBlock(64, 128)
        self.block4 = ConvBlock(128, 256)
        self.avg_pool = nn.AdaptiveAvgPool2d((1, 1))
        self.fc1 = nn.Linear(256, 128)
        self.relu_fc = nn.ReLU(inplace=False)
        self.fc2 = nn.Linear(128, num_classes)
        self.dequant = DeQuantStub()

    def forward(self, x):
        x = self.quant(x)
```

```
        x = self.block1(x)
        x = self.block2(x)
        x = self.block3(x)
        x = self.block4(x)
        x = self.avg_pool(x)
        x = torch.flatten(x, 1)
        x = self.relu_fc(self.fc1(x))
        x = self.fc2(x)
        return self.dequant(x)

    def fuse_model(self):
        self.block1.fuse_model()
        self.block2.fuse_model()
        self.block3.fuse_model()
        self.block4.fuse_model()
        fuse_modules(self, [["fc1", "relu_fc"]], inplace=True)


model = QuantizableSVHNCNN().to(TRAIN_DEVICE)
parameter_count = sum(p.numel() for p in model.parameters())
print(model)
print(f"\n模型参数量: {parameter_count:,}")
```

```
QuantizableSVHNCNN(
  (quant): QuantStub()
  (block1): ConvBlock(
    (conv1): Conv2d(3, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bia
s=False)
    (bn1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
    (relu1): ReLU()
    (conv2): Conv2d(32, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bi
as=False)
    (bn2): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
    (relu2): ReLU()
    (pool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=F
alse)
  )
  (block2): ConvBlock(
    (conv1): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bi
as=False)
    (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
    (relu1): ReLU()
    (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bi
as=False)
    (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
    (relu2): ReLU()
    (pool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=F
alse)
  )
  (block3): ConvBlock(
    (conv1): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), b
ias=False)
    (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_s
tats=True)
    (relu1): ReLU()
    (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
    (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_s
tats=True)
    (relu2): ReLU()
    (pool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=F
alse)
  )
  (block4): ConvBlock(
    (conv1): Conv2d(128, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
    (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_s
tats=True)
    (relu1): ReLU()
    (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
    (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_s
tats=True)
    (relu2): ReLU()
    (pool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=F
alse)
  )
  (avg_pool): AdaptiveAvgPool2d(output_size=(1, 1))
  (fc1): Linear(in_features=256, out_features=128, bias=True)
```

```
  (relu_fc): ReLU()
  (fc2): Linear(in_features=128, out_features=10, bias=True)
  (dequant): DeQuantStub()
)
```

模型参数量：1,207,402

4. FP32 模型训练或加载

本节优先加载已有权重。若权重不存在，则使用交叉熵损失、AdamW 优化器和余弦退火学习率训练模型，并保存测试准确率最高的参数。这样 notebook 可以独立复现实验，也可以直接复用已经训练好的基线模型。

In [5]:
```python
@torch.inference_mode()
def evaluate(model, loader, device):
    model.eval()
    correct = 0
    total = 0
    total_loss = 0.0
    criterion = nn.CrossEntropyLoss()

    for images, labels in loader:
        images = images.to(device)
        labels = labels.to(device)
        logits = model(images)
        total_loss += criterion(logits, labels).item() * labels.size(0)
        correct += (logits.argmax(dim=1) == labels).sum().item()
        total += labels.size(0)

    return total_loss / total, correct / total


def train_one_epoch(model, loader, criterion, optimizer, device):
    model.train()
    running_loss = 0.0
    correct = 0
    total = 0

    for images, labels in loader:
        images = images.to(device, non_blocking=True)
        labels = labels.to(device, non_blocking=True)

        optimizer.zero_grad(set_to_none=True)
        logits = model(images)
        loss = criterion(logits, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item() * labels.size(0)
        correct += (logits.argmax(dim=1) == labels).sum().item()
        total += labels.size(0)

    return running_loss / total, correct / total


def load_state_dict_compat(path, device):
    try:
        return torch.load(path, map_location=device, weights_only=True)
```

```
    except TypeError:
        return torch.load(path, map_location=device)
```

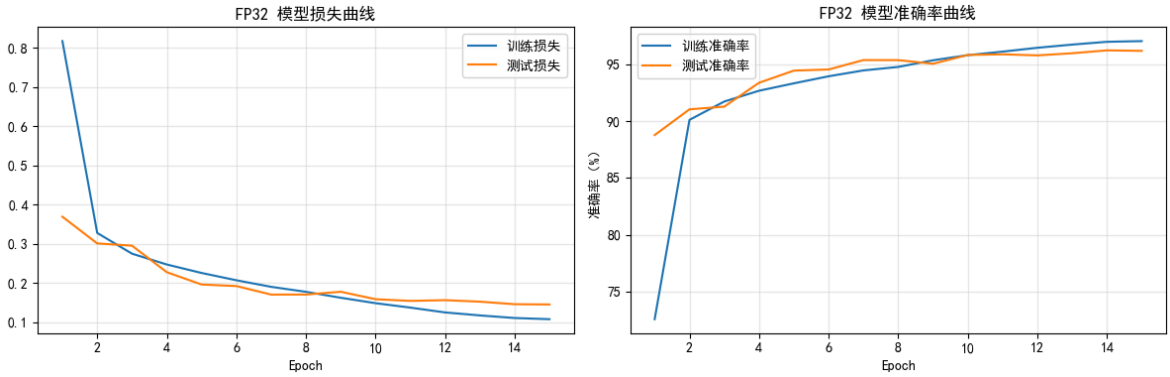
In [6]:

```
history = {"train_loss": [], "train_acc": [], "test_loss": [], "test_acc": []}

if CHECKPOINT_PATH.exists():
    model.load_state_dict(load_state_dict_compat(CHECKPOINT_PATH, TRAIN_DEVICE))
    print(f"已加载 FP32 权重：{CHECKPOINT_PATH}")
else:
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.AdamW(
        model.parameters(), lr=LEARNING_RATE, weight_decay=1e-4
    )
    scheduler = optim.lr_scheduler.CosineAnnealingLR(
        optimizer, T_max=NUM_EPOCHS
    )
    best_accuracy = 0.0

    for epoch in range(1, NUM_EPOCHS + 1):
        start = time.perf_counter()
        train_loss, train_acc = train_one_epoch(
            model, train_loader, criterion, optimizer, TRAIN_DEVICE
        )
        test_loss, test_acc = evaluate(model, test_loader, TRAIN_DEVICE)
        scheduler.step()

        history["train_loss"].append(train_loss)
        history["train_acc"].append(train_acc)
        history["test_loss"].append(test_loss)
        history["test_acc"].append(test_acc)

        if test_acc > best_accuracy:
            best_accuracy = test_acc
            torch.save(model.state_dict(), CHECKPOINT_PATH)

        print(
            f"Epoch {epoch:02d}/{NUM_EPOCHS} | "
            f"训练损失 {train_loss:.4f} | 训练准确率 {train_acc:.2%} | "
            f"测试损失 {test_loss:.4f} | 测试准确率 {test_acc:.2%} | "
            f"耗时 {time.perf_counter() - start:.1f}s"
        )

    model.load_state_dict(load_state_dict_compat(CHECKPOINT_PATH, TRAIN_DEVICE))
    print(f"训练完成，最佳 FP32 权重已保存至：{CHECKPOINT_PATH}")
```

```
Epoch 01/15 | 训练损失 0.8174 | 训练准确率 72.57% | 测试损失 0.3693 | 测试准确率 8
8.77% | 耗时 27.2s
Epoch 02/15 | 训练损失 0.3279 | 训练准确率 90.10% | 测试损失 0.3011 | 测试准确率 9
1.03% | 耗时 26.5s
Epoch 03/15 | 训练损失 0.2749 | 训练准确率 91.72% | 测试损失 0.2954 | 测试准确率 9
1.27% | 耗时 26.1s
Epoch 04/15 | 训练损失 0.2474 | 训练准确率 92.67% | 测试损失 0.2277 | 测试准确率 9
3.37% | 耗时 26.0s
Epoch 05/15 | 训练损失 0.2257 | 训练准确率 93.32% | 测试损失 0.1963 | 测试准确率 9
4.44% | 耗时 26.2s
Epoch 06/15 | 训练损失 0.2071 | 训练准确率 93.94% | 测试损失 0.1924 | 测试准确率 9
4.54% | 耗时 26.5s
Epoch 07/15 | 训练损失 0.1903 | 训练准确率 94.46% | 测试损失 0.1706 | 测试准确率 9
5.36% | 耗时 26.1s
Epoch 08/15 | 训练损失 0.1776 | 训练准确率 94.76% | 测试损失 0.1707 | 测试准确率 9
5.36% | 耗时 26.2s
Epoch 09/15 | 训练损失 0.1625 | 训练准确率 95.35% | 测试损失 0.1778 | 测试准确率 9
5.04% | 耗时 26.1s
Epoch 10/15 | 训练损失 0.1486 | 训练准确率 95.80% | 测试损失 0.1588 | 测试准确率 9
5.82% | 耗时 26.1s
Epoch 11/15 | 训练损失 0.1373 | 训练准确率 96.11% | 测试损失 0.1545 | 测试准确率 9
5.87% | 耗时 25.7s
Epoch 12/15 | 训练损失 0.1250 | 训练准确率 96.45% | 测试损失 0.1563 | 测试准确率 9
5.78% | 耗时 25.8s
Epoch 13/15 | 训练损失 0.1174 | 训练准确率 96.73% | 测试损失 0.1525 | 测试准确率 9
5.97% | 耗时 26.1s
Epoch 14/15 | 训练损失 0.1108 | 训练准确率 96.97% | 测试损失 0.1460 | 测试准确率 9
6.22% | 耗时 26.0s
Epoch 15/15 | 训练损失 0.1078 | 训练准确率 97.04% | 测试损失 0.1454 | 测试准确率 9
6.18% | 耗时 26.0s
训练完成，最佳 FP32 权重已保存至：svhn_fp32_cnn.pth
```

In [7]:

```python
if history["train_loss"]:
    epochs = np.arange(1, len(history["train_loss"]) + 1)
    fig, axes = plt.subplots(1, 2, figsize=(12, 4))
    axes[0].plot(epochs, history["train_loss"], label="训练损失")
    axes[0].plot(epochs, history["test_loss"], label="测试损失")
    axes[0].set_title("FP32 模型损失曲线")
    axes[0].set_xlabel("Epoch")
    axes[0].legend()
    axes[0].grid(alpha=0.3)

    axes[1].plot(epochs, np.array(history["train_acc"]) * 100, label="训练准确率
    axes[1].plot(epochs, np.array(history["test_acc"]) * 100, label="测试准确率"
    axes[1].set_title("FP32 模型准确率曲线")
    axes[1].set_xlabel("Epoch")
    axes[1].set_ylabel("准确率（%）")
    axes[1].legend()
    axes[1].grid(alpha=0.3)
    plt.tight_layout()
    plt.show()
else:
    print("本次直接加载已有权重，因此不显示训练曲线。")
```



5. 手动实现 per-tensor 非对称线性量化

对于浮点张量 x，将其映射到 b 位无符号整数区间：

$$q_{\min} = 0, \qquad q_{\max} = 2^b - 1$$

量化尺度与零点为：

$$s = \frac{x_{\max} - x_{\min}}{q_{\max} - q_{\min}}$$

$$z = \mathrm{clip}\left(\mathrm{round}\left(q_{\min} - \frac{x_{\min}}{s}\right), q_{\min}, q_{\max}\right)$$

量化和反量化公式分别为：

$$q = \mathrm{clip}\left(\mathrm{round}\left(\frac{x}{s} + z\right), q_{\min}, q_{\max}\right)$$

$$\hat{x} = s(q - z)$$

下面两个函数完全按照上述公式手动实现，没有调用 PyTorch 的量化 API。

In [8]:
```python
def linear_quantize(x, num_bits=8):
    # 将浮点张量进行 per-tensor 非对称线性量化。
    if not torch.is_floating_point(x):
        raise TypeError("输入 x 必须是浮点张量")
    if not 2 <= num_bits <= 16:
        raise ValueError("num_bits 应位于 [2, 16] 区间")

    q_min = 0
    q_max = 2**num_bits - 1
    x_min = x.detach().min()
    x_max = x.detach().max()

    value_range = x_max - x_min
    scale = value_range / float(q_max - q_min)
    if scale.item() == 0:
        scale = torch.tensor(1.0, device=x.device, dtype=x.dtype)

    zero_point = q_min - torch.round(x_min / scale)
    zero_point = zero_point.clamp(q_min, q_max).to(torch.int64)
    q = torch.round(x / scale + zero_point).clamp(q_min, q_max)
    q_dtype = torch.uint8 if num_bits <= 8 else torch.int32
    return q.to(q_dtype), scale.detach(), zero_point.detach()
```

```
def linear_dequantize(q, scale, zero_point):
    # 根据尺度和零点将整数张量反量化为浮点张量。
    return scale * (q.to(torch.float32) - zero_point.to(torch.float32))
```

In [9]:
```
# 用随机张量验证手写量化函数
demo_x = torch.randn(4, 8) * 2.5 - 0.7
demo_q, demo_scale, demo_zero_point = linear_quantize(demo_x, num_bits=8)
demo_x_hat = linear_dequantize(demo_q, demo_scale, demo_zero_point)
demo_mse = torch.mean((demo_x - demo_x_hat) ** 2).item()

assert demo_q.dtype == torch.uint8
assert 0 <= demo_q.min() and demo_q.max() <= 255

print("原浮点范围：", (demo_x.min().item(), demo_x.max().item()))
print("INT8 范围：", (demo_q.min().item(), demo_q.max().item()))
print(f"scale = {demo_scale.item():.8f}")
print(f"zero_point = {demo_zero_point.item()}")
print(f"反量化 MSE = {demo_mse:.10f}")
```

```
原浮点范围： (-4.711520195007324, 5.489171028137207)
INT8 范围： (0, 255)
scale = 0.04000271
zero_point = 118
反量化 MSE = 0.0001253175
```

6. FP32 基线指标

为公平比较，量化前后模型均在 CPU 上测试。模型大小通过序列化 `state_dict` 后的字节数计算；单张图像延迟先预热，再重复推理并取算术平均值。

In [10]:
```
def serialized_model_size_mb(model):
    buffer = io.BytesIO()
    torch.save(model.state_dict(), buffer)
    return buffer.getbuffer().nbytes / (1024**2)


@torch.inference_mode()
def measure_single_image_latency(model, sample, warmup=30, repeats=300):
    model.eval()
    sample = sample.cpu()

    for _ in range(warmup):
        model(sample)

    latencies_ms = []
    for _ in range(repeats):
        start = time.perf_counter()
        model(sample)
        latencies_ms.append((time.perf_counter() - start) * 1000)

    return float(np.mean(latencies_ms)), float(np.std(latencies_ms))


fp32_model = copy.deepcopy(model).to(CPU_DEVICE).eval()
fp32_loss, fp32_accuracy = evaluate(fp32_model, test_loader, CPU_DEVICE)
sample_image = next(iter(test_loader))[0][:1].cpu()
```

```
fp32_size_mb = serialized_model_size_mb(fp32_model)
fp32_latency_ms, fp32_latency_std = measure_single_image_latency(
    fp32_model, sample_image
)

print(f"FP32 测试损失：{fp32_loss:.4f}")
print(f"FP32 测试准确率：{fp32_accuracy:.2%}")
print(f"FP32 模型大小：{fp32_size_mb:.3f} MB")
print(
    f"FP32 CPU 单张延迟：{fp32_latency_ms:.3f} ± "
    f"{fp32_latency_std:.3f} ms"
)
```

```
FP32 测试损失：0.1460
FP32 测试准确率：96.22%
FP32 模型大小：4.629 MB
FP32 CPU 单张延迟：2.083 ± 0.248 ms
```

7. INT8 静态量化（PTQ）

静态量化流程如下：

1. 复制训练好的 FP32 模型并切换到 CPU 推理模式；
2. 融合卷积、批归一化和 ReLU，以及全连接层和 ReLU；
3. 配置激活与权重观察器，二者均采用 per-tensor 非对称线性量化；
4. 使用无标签训练样本前向传播，收集激活值范围并完成校准；
5. 调用 `convert` 将带观察器的模型转换为真正的 INT8 推理模型。

权重使用有符号 `qint8`，激活使用无符号 `quint8`；两者的 `qscheme` 都是 `per_tensor_affine`。

In [11]:
```
supported_engines = torch.backends.quantized.supported_engines
if "x86" in supported_engines:
    QUANT_ENGINE = "x86"
elif "fbgemm" in supported_engines:
    QUANT_ENGINE = "fbgemm"
elif "qnnpack" in supported_engines:
    QUANT_ENGINE = "qnnpack"
else:
    raise RuntimeError(f"未找到可用的 PyTorch 量化后端：{supported_engines}")

torch.backends.quantized.engine = QUANT_ENGINE

activation_observer = MinMaxObserver.with_args(
    dtype=torch.quint8,
    qscheme=torch.per_tensor_affine,
    quant_min=0,
    quant_max=255,
)
weight_observer = MinMaxObserver.with_args(
    dtype=torch.qint8,
    qscheme=torch.per_tensor_affine,
    quant_min=-128,
    quant_max=127,
)
ptq_qconfig = QConfig(
    activation=activation_observer,
```

```
    weight=weight_observer,
)

fused_fp32_model = copy.deepcopy(fp32_model).eval()
fused_fp32_model.fuse_model()

prepared_model = copy.deepcopy(fused_fp32_model)
prepared_model.qconfig = ptq_qconfig
prepare(prepared_model, inplace=True)

print("量化后端：", QUANT_ENGINE)
print("量化配置：", prepared_model.qconfig)
print("\n融合并插入观察器后的模型：")
print(prepared_model)
```

量化后端： x86
量化配置： QConfig(activation=functools.partial(<class 'torch.ao.quantization.observer.MinMaxObserver'>, dtype=torch.quint8, qscheme=torch.per_tensor_affine, quant_min=0, quant_max=255){'factory_kwargs': <function _add_module_to_qconfig_obs_ctr.<locals>.get_factory_kwargs_based_on_module_device at 0x000001862FA67380>}, weight=functools.partial(<class 'torch.ao.quantization.observer.MinMaxObserver'>, dtype=torch.qint8, qscheme=torch.per_tensor_affine, quant_min=-128, quant_max=127){'factory_kwargs': <function _add_module_to_qconfig_obs_ctr.<locals>.get_factory_kwargs_based_on_module_device at 0x000001862FA67380>})

融合并插入观察器后的模型：

```
QuantizableSVHNCNN(
  (quant): QuantStub(
    (activation_post_process): MinMaxObserver(min_val=inf, max_val=-inf)
  )
  (block1): ConvBlock(
    (conv1): ConvReLU2d(
      (0): Conv2d(3, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
      (1): ReLU()
      (activation_post_process): MinMaxObserver(min_val=inf, max_val=-inf)
    )
    (bn1): Identity()
    (relu1): Identity()
    (conv2): ConvReLU2d(
      (0): Conv2d(32, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
      (1): ReLU()
      (activation_post_process): MinMaxObserver(min_val=inf, max_val=-inf)
    )
    (bn2): Identity()
    (relu2): Identity()
    (pool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=F
alse)
  )
  (block2): ConvBlock(
    (conv1): ConvReLU2d(
      (0): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
      (1): ReLU()
      (activation_post_process): MinMaxObserver(min_val=inf, max_val=-inf)
    )
    (bn1): Identity()
    (relu1): Identity()
    (conv2): ConvReLU2d(
      (0): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
      (1): ReLU()
      (activation_post_process): MinMaxObserver(min_val=inf, max_val=-inf)
    )
    (bn2): Identity()
    (relu2): Identity()
    (pool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=F
alse)
  )
  (block3): ConvBlock(
    (conv1): ConvReLU2d(
      (0): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
      (1): ReLU()
      (activation_post_process): MinMaxObserver(min_val=inf, max_val=-inf)
    )
    (bn1): Identity()
    (relu1): Identity()
    (conv2): ConvReLU2d(
```

```
      (0): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
      (1): ReLU()
      (activation_post_process): MinMaxObserver(min_val=inf, max_val=-inf)
    )
    (bn2): Identity()
    (relu2): Identity()
    (pool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=F
alse)
  )
  (block4): ConvBlock(
    (conv1): ConvReLU2d(
      (0): Conv2d(128, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
      (1): ReLU()
      (activation_post_process): MinMaxObserver(min_val=inf, max_val=-inf)
    )
    (bn1): Identity()
    (relu1): Identity()
    (conv2): ConvReLU2d(
      (0): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
      (1): ReLU()
      (activation_post_process): MinMaxObserver(min_val=inf, max_val=-inf)
    )
    (bn2): Identity()
    (relu2): Identity()
    (pool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=F
alse)
  )
  (avg_pool): AdaptiveAvgPool2d(output_size=(1, 1))
  (fc1): LinearReLU(
    (0): Linear(in_features=256, out_features=128, bias=True)
    (1): ReLU()
    (activation_post_process): MinMaxObserver(min_val=inf, max_val=-inf)
  )
  (relu_fc): Identity()
  (fc2): Linear(
    in_features=128, out_features=10, bias=True
    (activation_post_process): MinMaxObserver(min_val=inf, max_val=-inf)
  )
  (dequant): DeQuantStub()
)
```

```
C:\Users\A\AppData\Local\Temp\ipykernel_29664\2693116813.py:35: DeprecationWarnin
g: torch.ao.quantization is deprecated and will be removed in 2.10.
For migrations of users:
1. Eager mode quantization (torch.ao.quantization.quantize, torch.ao.quantizatio
n.quantize_dynamic), please migrate to use torchao eager mode quantize_ API inste
ad
2. FX graph mode quantization (torch.ao.quantization.quantize_fx.prepare_fx,torc
h.ao.quantization.quantize_fx.convert_fx, please migrate to use torchao pt2e quan
tization API instead (prepare_pt2e, convert_pt2e)
3. pt2e quantization has been migrated to torchao (https://github.com/pytorch/ao/
tree/main/torchao/quantization/pt2e)
see https://github.com/pytorch/ao/issues/2259 for more details
  prepare(prepared_model, inplace=True)
```

In [12]:

```python
@torch.inference_mode()
def calibrate(model, loader):
    model.eval()
    sample_count = 0
    for images, _ in loader:
        model(images.cpu())
```

```
        sample_count += images.size(0)
    return sample_count


calibrated_count = calibrate(prepared_model, calibration_loader)
int8_model = convert(prepared_model, inplace=False).eval()

print(f"完成校准，使用无标签样本数：{calibrated_count}")
print("\n转换后的 INT8 模型：")
print(int8_model)
```

完成校准，使用无标签样本数：1024

转换后的 INT8 模型：

```
QuantizableSVHNCNN(
  (quant): Quantize(scale=tensor([0.0205]), zero_point=tensor([117]), dtype=torc
h.quint8)
  (block1): ConvBlock(
    (conv1): QuantizedConvReLU2d(3, 32, kernel_size=(3, 3), stride=(1, 1), scale=
0.024054430425167084, zero_point=0, padding=(1, 1))
    (bn1): Identity()
    (relu1): Identity()
    (conv2): QuantizedConvReLU2d(32, 32, kernel_size=(3, 3), stride=(1, 1), scale
=0.020183205604553223, zero_point=0, padding=(1, 1))
    (bn2): Identity()
    (relu2): Identity()
    (pool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=F
alse)
  )
  (block2): ConvBlock(
    (conv1): QuantizedConvReLU2d(32, 64, kernel_size=(3, 3), stride=(1, 1), scale
=0.02587796375155449, zero_point=0, padding=(1, 1))
    (bn1): Identity()
    (relu1): Identity()
    (conv2): QuantizedConvReLU2d(64, 64, kernel_size=(3, 3), stride=(1, 1), scale
=0.025912923738360405, zero_point=0, padding=(1, 1))
    (bn2): Identity()
    (relu2): Identity()
    (pool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=F
alse)
  )
  (block3): ConvBlock(
    (conv1): QuantizedConvReLU2d(64, 128, kernel_size=(3, 3), stride=(1, 1), scal
e=0.026664189994335175, zero_point=0, padding=(1, 1))
    (bn1): Identity()
    (relu1): Identity()
    (conv2): QuantizedConvReLU2d(128, 128, kernel_size=(3, 3), stride=(1, 1), sca
le=0.030071290209889412, zero_point=0, padding=(1, 1))
    (bn2): Identity()
    (relu2): Identity()
    (pool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=F
alse)
  )
  (block4): ConvBlock(
    (conv1): QuantizedConvReLU2d(128, 256, kernel_size=(3, 3), stride=(1, 1), sca
le=0.035088568925857544, zero_point=0, padding=(1, 1))
    (bn1): Identity()
    (relu1): Identity()
    (conv2): QuantizedConvReLU2d(256, 256, kernel_size=(3, 3), stride=(1, 1), sca
le=0.04567839577794075, zero_point=0, padding=(1, 1))
    (bn2): Identity()
    (relu2): Identity()
    (pool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=F
alse)
  )
  (avg_pool): AdaptiveAvgPool2d(output_size=(1, 1))
  (fc1): QuantizedLinearReLU(in_features=256, out_features=128, scale=0.090428546
07105255, zero_point=0, qscheme=torch.per_tensor_affine)
  (relu_fc): Identity()
  (fc2): QuantizedLinear(in_features=128, out_features=10, scale=0.10685975104570
389, zero_point=93, qscheme=torch.per_tensor_affine)
```

```
  (dequant): DeQuantize()
)
C:\Users\A\AppData\Local\Temp\ipykernel_29664\1169021643.py:12: DeprecationWarnin
g: torch.ao.quantization is deprecated and will be removed in 2.10.
For migrations of users:
1. Eager mode quantization (torch.ao.quantization.quantize, torch.ao.quantizatio
n.quantize_dynamic), please migrate to use torchao eager mode quantize_ API inste
ad
2. FX graph mode quantization (torch.ao.quantization.quantize_fx.prepare_fx,torc
h.ao.quantization.quantize_fx.convert_fx, please migrate to use torchao pt2e quan
tization API instead (prepare_pt2e, convert_pt2e)
3. pt2e quantization has been migrated to torchao (https://github.com/pytorch/ao/
tree/main/torchao/quantization/pt2e)
see https://github.com/pytorch/ao/issues/2259 for more details
  int8_model = convert(prepared_model, inplace=False).eval()
```

8. 各层输出量化 MSE

通过前向钩子同时记录融合 FP32 模型和 INT8 模型中主要层的输出。对于 INT8 中间张量，先使用其量化参数反量化，再与对应 FP32 输出计算均方误差：

$$\mathrm{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i^{FP32} - \hat{y}_i^{INT8})^2$$

这里比较四个卷积块、全局平均池化层、融合后的第一全连接层以及最终分类层。

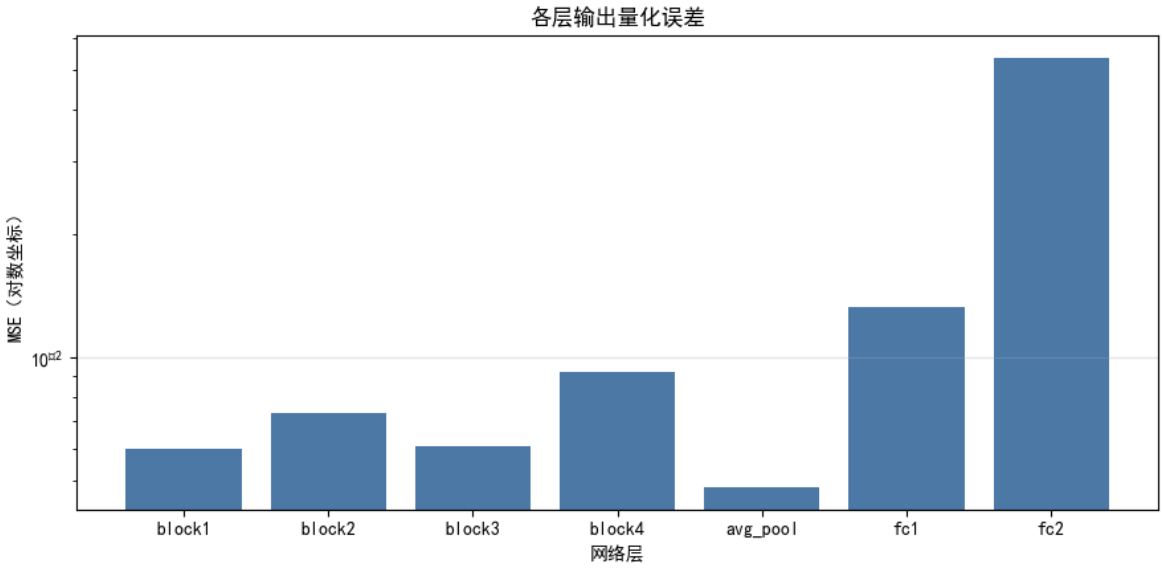
In [13]:
```python
LAYER_NAMES = [
    "block1", "block2", "block3", "block4",
    "avg_pool", "fc1", "fc2",
]


def capture_layer_outputs(model, layer_names, inputs):
    outputs = {}
    handles = []

    def make_hook(name):
        def hook(_module, _inputs, output):
            tensor = output[0] if isinstance(output, (tuple, list)) else output
            if tensor.is_quantized:
                tensor = tensor.dequantize()
            outputs[name] = tensor.detach().cpu().float()
        return hook

    modules = dict(model.named_modules())
    for name in layer_names:
        if name not in modules:
            raise KeyError(f"模型中不存在待比较层：{name}")
        handles.append(modules[name].register_forward_hook(make_hook(name)))

    with torch.inference_mode():
        model(inputs.cpu())

    for handle in handles:
        handle.remove()
    return outputs
```

```
mse_images = next(iter(test_loader))[0][:64].cpu()
fp32_layer_outputs = capture_layer_outputs(
    fused_fp32_model, LAYER_NAMES, mse_images
)
int8_layer_outputs = capture_layer_outputs(
    int8_model, LAYER_NAMES, mse_images
)

layer_mse = {
    name: torch.mean(
        (fp32_layer_outputs[name] - int8_layer_outputs[name]) ** 2
    ).item()
    for name in LAYER_NAMES
}

mse_table = pd.DataFrame({
    "层名称": list(layer_mse.keys()),
    "输出量化 MSE": list(layer_mse.values()),
})
display(mse_table.style.format({"输出量化 MSE": "{:.8e}"}))
```

	层名称	输出量化 MSE
0	block1	5.97999478e-03
1	block2	7.31672347e-03
2	block3	6.08937675e-03
3	block4	9.17488895e-03
4	avg_pool	4.80980193e-03
5	fc1	1.32393297e-02
6	fc2	5.37247248e-02

In [14]:
```
plt.figure(figsize=(9, 4.5))
plt.bar(mse_table["层名称"], mse_table["输出量化 MSE"], color="#4C78A8")
plt.yscale("log")
plt.xlabel("网络层")
plt.ylabel("MSE（对数坐标）")
plt.title("各层输出量化误差")
plt.grid(axis="y", alpha=0.3)
plt.tight_layout()
plt.show()
```

```
Font 'default' does not have a glyph for '\u2212' [U+2212], substituting with a d
ummy symbol.
Font 'default' does not have a glyph for '\u2212' [U+2212], substituting with a d
ummy symbol.
Font 'default' does not have a glyph for '\u2212' [U+2212], substituting with a d
ummy symbol.
Font 'default' does not have a glyph for '\u2212' [U+2212], substituting with a d
ummy symbol.
```



9. INT8 模型评估与综合对比

本节在完整测试集上评估 INT8 模型，并计算：

- 精度损失 = FP32 准确率 − INT8 准确率；
- 压缩比 = FP32 模型大小 / INT8 模型大小；
- 加速比 = FP32 平均延迟 / INT8 平均延迟。

In [15]:

```
int8_loss, int8_accuracy = evaluate(int8_model, test_loader, CPU_DEVICE)
int8_size_mb = serialized_model_size_mb(int8_model)
int8_latency_ms, int8_latency_std = measure_single_image_latency(
    int8_model, sample_image
)

accuracy_loss = fp32_accuracy - int8_accuracy
compression_ratio = fp32_size_mb / int8_size_mb
speedup_ratio = fp32_latency_ms / int8_latency_ms

comparison = pd.DataFrame({
    "模型": ["FP32", "INT8"],
    "测试准确率（%）": [fp32_accuracy * 100, int8_accuracy * 100],
    "模型大小（MB）": [fp32_size_mb, int8_size_mb],
    "CPU 单张平均延迟（ms）": [fp32_latency_ms, int8_latency_ms],
})

display(comparison.style.format({
    "测试准确率（%）": "{:.3f}",
    "模型大小（MB）": "{:.3f}",
    "CPU 单张平均延迟（ms）": "{:.3f}",
}))

print(f"量化精度损失：{accuracy_loss * 100:.3f} 个百分点")
print(f"模型压缩比：{compression_ratio:.2f} 倍")
print(f"CPU 推理加速比：{speedup_ratio:.2f} 倍")
```

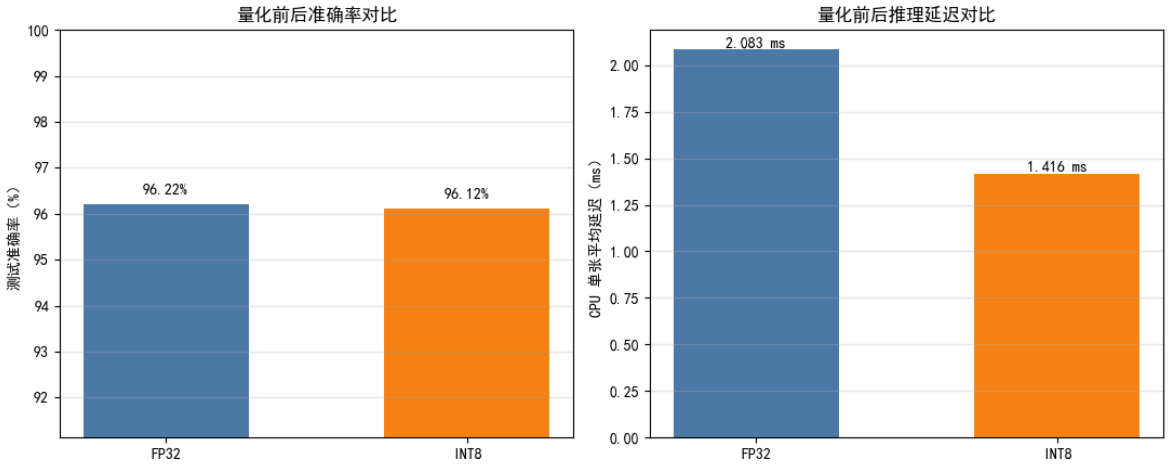
	模型	测试准确率（%）	模型大小（MB）	CPU 单张平均延迟（ms）
0	FP32	96.220	4.629	2.083
1	INT8	96.124	1.167	1.416

```
量化精度损失：0.096 个百分点
模型压缩比：3.97 倍
CPU 推理加速比：1.47 倍
```

10. 可视化结果

按照作业要求，分别绘制量化前后准确率对比图和 CPU 单张图像平均推理延迟对比图。延迟测试结果与 CPU 型号、线程数及 PyTorch 版本有关，因此报告时应以本机实际运行结果为准。

In [16]:
```python
colors = ["#4C78A8", "#F58518"]
fig, axes = plt.subplots(1, 2, figsize=(11, 4.5))

acc_values = [fp32_accuracy * 100, int8_accuracy * 100]
bars_acc = axes[0].bar(["FP32", "INT8"], acc_values, color=colors, width=0.55)
axes[0].set_ylabel("测试准确率（%）")
axes[0].set_title("量化前后准确率对比")
axes[0].set_ylim(max(0, min(acc_values) - 5), 100)
axes[0].grid(axis="y", alpha=0.3)
for bar, value in zip(bars_acc, acc_values):
    axes[0].text(
        bar.get_x() + bar.get_width() / 2, value + 0.2,
        f"{value:.2f}%", ha="center"
    )

latency_values = [fp32_latency_ms, int8_latency_ms]
bars_latency = axes[1].bar(
    ["FP32", "INT8"], latency_values, color=colors, width=0.55
)
axes[1].set_ylabel("CPU 单张平均延迟（ms）")
axes[1].set_title("量化前后推理延迟对比")
axes[1].grid(axis="y", alpha=0.3)
for bar, value in zip(bars_latency, latency_values):
    axes[1].text(
        bar.get_x() + bar.get_width() / 2, value,
        f"{value:.3f} ms", ha="center", va="bottom"
    )

plt.tight_layout()
plt.savefig("quantization_comparison.png", dpi=180, bbox_inches="tight")
plt.show()
```



In [17]:
```python
# 可视化 FP32 与 INT8 的部分预测结果
show_images, show_labels = next(iter(test_loader))
show_images = show_images[:16].cpu()
show_labels = show_labels[:16]

with torch.inference_mode():
    fp32_preds = fp32_model(show_images).argmax(dim=1)
    int8_preds = int8_model(show_images).argmax(dim=1)

fig, axes = plt.subplots(2, 8, figsize=(14, 5))
for i, ax in enumerate(axes.flat):
    image = (show_images[i] * std + mean).permute(1, 2, 0).clamp(0, 1)
    correct = int8_preds[i].item() == show_labels[i].item()
    ax.imshow(image)
    ax.set_title(
        f"真:{show_labels[i].item()} "
        f"F:{fp32_preds[i].item()} I:{int8_preds[i].item()}",
        color="green" if correct else "red",
        fontsize=9,
    )
    ax.axis("off")
plt.suptitle("预测结果对比（F=FP32，I=INT8）")
plt.tight_layout()
plt.show()
```

11. 实验结论

下方代码根据本次实际运行结果自动生成结论，避免手工填写时出现数值不一致。

In [18]:
```python
print("实验结论：")
print(
```

```
    f"1. FP32 模型测试准确率为 {fp32_accuracy:.2%}，"
    f"INT8 模型测试准确率为 {int8_accuracy:.2%}，"
    f"精度损失为 {accuracy_loss * 100:.3f} 个百分点。"
)
print(
    f"2. 模型大小由 {fp32_size_mb:.3f} MB 降至 {int8_size_mb:.3f} MB，"
    f"压缩比为 {compression_ratio:.2f} 倍。"
)
print(
    f"3. CPU 单张图像平均推理延迟由 {fp32_latency_ms:.3f} ms "
    f"降至 {int8_latency_ms:.3f} ms，加速比为 {speedup_ratio:.2f} 倍。"
)
worst_layer = max(layer_mse, key=layer_mse.get)
print(
    f"4. 各主要层均存在一定量化误差，其中本批样本上 MSE 最大的层为 "
    f"{worst_layer}，MSE={layer_mse[worst_layer]:.8e}。"
)
print(
    "5. 综上，静态 INT8 量化通常能在仅产生较小精度损失的前提下，"
    "显著减小模型体积并改善 CPU 推理效率，适合资源受限的边缘部署场景。"
)
```

```
实验结论：
1. FP32 模型测试准确率为 96.22%，INT8 模型测试准确率为 96.12%，精度损失为 0.096 个百分点。
2. 模型大小由 4.629 MB 降至 1.167 MB，压缩比为 3.97 倍。
3. CPU 单张图像平均推理延迟由 2.083 ms 降至 1.416 ms，加速比为 1.47 倍。
4. 各主要层均存在一定量化误差，其中本批样本上 MSE 最大的层为 fc2，MSE=5.37247248e-02。
5. 综上，静态 INT8 量化通常能在仅产生较小精度损失的前提下，显著减小模型体积并改善 CPU 推理效率，适合资源受限的边缘部署场景。
```

12. 保存 INT8 模型

保存量化模型参数，便于后续在相同 PyTorch 版本、相同量化后端的 CPU 环境中加载。量化模型不应移动到 CUDA 上运行。

In [19]:
```
INT8_CHECKPOINT_PATH = Path("./svhn_int8_static_quantized.pth")
torch.save(
    {
        "state_dict": int8_model.state_dict(),
        "quant_engine": QUANT_ENGINE,
        "qscheme": "per_tensor_affine",
        "fp32_accuracy": fp32_accuracy,
        "int8_accuracy": int8_accuracy,
    },
    INT8_CHECKPOINT_PATH,
)
print(f"INT8 模型已保存至：{INT8_CHECKPOINT_PATH}")
```

```
INT8 模型已保存至：svhn_int8_static_quantized.pth
```